

get

get

```
std::get
```

```
template <typename... Ts>
struct WeirdTuple : std::tuple<Ts...> {
    using std::tuple<Ts...>::tuple;
};

template <std::size_t I, typename... Ts>
auto get(WeirdTuple<Ts...>& t) { return I + 10; }

int main() {
    WeirdTuple<int> wt(1);

    get<0>(wt); // changing this 0 to anything else will break
                // theoretically, that should still work -
                // we're obviously using the WeirdTuple overload of get<>(),
                // but the compiler fails when trying to compile every possible overload!
}
```

```
get<1>(wt); // error: static assertion failed: tuple index is in range
            // in instantiation of 'std::tuple_element<1, std::tuple<int>>'
            // required by substitution of
            // constexpr std::tuple_element_t<I, std::tuple<Ts...>&
            // std::get(std::tuple<Ts...>&)
            // [with I = 1; Ts = {int}]
```

```
std::get
```

WeirdTuple

std::get

 t

```
std::tuple<UDT> get<1>(t)
    UDT
```

```
get
std::get
```

```
template <std::size_t I, typename ...Ts,
    typename Enable = std::enable_if_t<I < sizeof...(Ts)>>
std::tuple_element_t<I, std::tuple<Ts...>>&
get(std::tuple<Ts...>& t) {
    return /*...*/;
}
```

```
std::tuple<int> t;
std::get<1>(t); // error: no matching function for call to 'get<1>(std::tuple<int>&)'
                // note: candidate template ignored:
                //   std::get(array<Ts...>&)
                //   could not match 'array' against 'tuple'
                // note: candidate template ignored:
                //   std::get(pair<Ts...>&)
                //   could not match 'pair' against 'tuple'
                // note: candidate template ignored:
                //   std::get(variant<Ts...>&)
                //   could not match 'variant' against 'tuple'
                // note: candidate template ignored:
                //   std::get(std::tuple<Ts...>&)
                //   [with I = 1; Ts = {int}]
                //   requirement 'I < sizeof...(Ts)' was not satisfied
```

```
template <std::size_t I, typename ...Ts,
    typename Enable = std::enable_if_t<I < sizeof...(Ts)>>
std::tuple_element_t<I, std::tuple<Ts...>>&
get(std::tuple<Ts...>& t) {
    return /*...*/;
}

template <std::size_t I, typename ...Ts>
std::enable_if_t<sizeof...(Ts) <= I>
get(std::tuple<Ts...>& t) = delete;
```

```
std::tuple<int> t;
std::get<1>(t); // error: call to deleted function 'get'
                // note: declared here
                //   std::get(std::tuple<Ts...>&) = delete;
                //   [with I = 1; Ts = {int}]
```

```
template <std::size_t I, typename ...Ts>
decltype(auto) get(std::tuple<Ts...>& t) {
    static_assert(I < sizeof...(Ts), "tuple index is in range");
    return /*...*/;
}
```

```
std::tuple<int> t;
std::get<1>(t); // error: static assertion failed: tuple index is in range
               // in instantiation of
               //   std::get(std::tuple<Ts...>&)
               //   [with I = 1; Ts = {int}]
```

```
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>>&
get(tuple<Types...> t) noexcept;
```

~~I < sizeof...(Types)~~

I < sizeof...(Types)

```
template <class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
```

~~T Types...~~

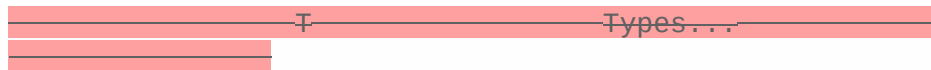
T Types...

```
template <size_t I, class... Types>
constexpr tuple_element_t<I, tuple<Types...>>&
get(tuple<Types...> t) noexcept;
```

~~I < sizeof...(Types)~~

I < sizeof...(Types)

```
template <class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
```



```
template <size_t I, class... Types>  
constexpr tuple_element_t<I, tuple<Types...>>& decltype(auto)  
get(tuple<Types...>& t) noexcept;
```

I < sizeof...(Types)

```
template <class T, class... Types>  
constexpr T& decltype(auto) get(tuple<Types...>& t) noexcept;
```

T

Types...

-

-

tuple_element variant_alternative